One in seventy decisions depended on dictionary insertion order: a determinism audit of a multi-engine OCR ensemble

Igor Saevets
https://github.com/igorsaevets/scanquorum

August 2026

Abstract

Combining several OCR engines by voting is more than thirty years old. We report a permutation-order determinism audit of one such system. We have found no prior report of this test on an OCR voting ensemble, and we do not claim priority. We generated a fixture of 11,780 distinct decisions from a running ensemble, replayed each one under every permutation of the order in which the engines are supplied, and required the set of outcomes to have size one. It did not. On 165 decisions, or 1.40 percent, the answer depended on which engine the underlying dictionary happened to yield first. Twenty nine of those changed a digit. In a corpus of legal citations the difference between `12-869` and `12369` is the difference between two cases. The cause was mundane, and can affect any ensemble that uses first-encountered order as an unstated tie break: `Counter.most_common()` and `max()` resolve a tie by returning whichever candidate was inserted first. We describe the four sites and their fix. We also describe a fifth site outside the voter that the permutation test could not reach; an external reviewer found it. The audited system treats refusal as an output rather than a failure. We report its measured accuracy, and two reasons that number is an upper bound rather than an estimate. Code, the reference standard and the replay fixture are public.

1 Introduction

Wrong characters are cheap. What costs money is a wrong character that nothing downstream can tell apart from a right one.

This has become sharper with retrieval augmented generation. A pipeline that runs OCR over a scanned filing and hands the result to a language model will reproduce a misread docket number with the same fluency, and the same absence of hedging, as a correct one. Unless the pipeline propagates confidence, disagreement or refusal, the model receives the text as though every token were equally reliable. By the time it arrives, the distinction between a corroborated reading and a guess has usually been discarded, and nothing downstream can act on a difference it was never told about.

The system described here takes a narrow position on that problem. It does not try to read scans more accurately than the best available engine. It tries to make every token in its output carry a statement about how it was decided. Where its engines disagree it either flags the reading or refuses to emit one, and lists the disagreement either way.

That design is not novel and we do not claim it is. Section 2 sets out how much of it is thirty years old. What we do claim is the finding in Section 4, and one methodological point: an ensemble whose tie breaks are resolved by an unspecified rule is not reproducibly specified. The same inputs can yield different outputs when insertion order changes, and none of the OCR ensemble papers we cite reports a check for it.

2 Related work

Voting across recognisers. ROVER [4] aligns the output of several speech recognisers into a word transition network and votes over it, optionally weighting by each recogniser's confidence. The technique moved into document recognition quickly. The ISRI toolkit at UNLV [11, 9] shipped `synctext` for alignment and `vote` for combination, and the documentation of `vote` records that it applies majority voting with heuristics to break ties [9]. The problem in Section 4 was therefore visible in that documentation to anyone who asked what those heuristics were, and we have found no report that anyone did.

Lund and Ringger [7] align several engines at the character level using an admissible A* heuristic and select from the resulting lattice with a dictionary, reporting a 35.8 percent reduction in word error rate on a corpus of typewritten communiqués. Lund, Walker and Ringger [8] replace the selection step with a discriminative model and report 24.6 percent relative improvement over the best single engine. Handley [6] had already surveyed the space and separated classifier level from string level combination.

Voting inside one engine. Calamari [12] votes across models of a single architecture trained on different folds, and reaches character error rates well below anything we report, on cleaner material. Reul and colleagues [10] apply the same idea to early printed books. This line of work is a genuine alternative to ours rather than a predecessor: it gets its diversity from model instances, and we get ours from differing architectures. Section 3.1 gives our reason for preferring the second, which is a measurement rather than an argument.

Abstention. Refusing to answer is not new either, and the literature is considerably better developed than our use of it. Chow [1, 2] formulated the reject option and derived the optimal threshold from the relative cost of an error and a rejection. Modern selective prediction [3, 5] reports a risk coverage curve rather than a single operating point. We report a single operating point. We have no calibrated confidence to threshold, and the acceptance rule is fixed rather than tunable, so a curve is not currently available to us. This is a limitation and Section 6 treats it as one.

Ensembles as working software. OCR-D's alignment step performs n-ary alignment with majority or confidence voting as an operational component of a document processing pipeline. Multi-engine OCR as usable software is not new, and a comparison against that baseline is the most obvious experiment we have not run.

3 The system

Four voters read each page. One is the text layer already present in the PDF, whoever produced it. Two are RapidOCR with a shared PP-OCRv6 detector and different recognisers. The fourth is Tesseract 5.

Word boxes come from a frame engine chosen by a fixed preference order. Each voter's reading is aligned to that frame, and a cascade of rules decides the token. The rules, in order, are: exactly one engine produced anything; two or more produced the same normalised string; the engines agree on the digits and differ on a separator; a numeric token where a majority agree on the digit sequence; one candidate is in the lexicon and the others are not; one candidate matches a citation pattern; no majority, in which case the medoid is reported and flagged; and finally no defensible choice, in which case the system refuses.

One property holds across all of them. Every rule returns a string that some engine actually proposed, or nothing. No rule composes a new string from parts of several readings. This is

weaker than it sounds, because simple string level voters share it; lattice and language model based reconciliation does not.

The output is a Markdown file whose header states, in fields that sum to the token count, how many words were read identically by two or more independent engines, how many agreed on digits but not punctuation, how many were settled by dictionary or pattern with no second engine agreeing, and how many were not confirmed at all. The last group is written to a separate file with page and coordinates.

3.1 Independence is not the engine count

Two of the four voters share a text detector. On one page set, 2,921 of 3,006 of their boxes were identical. Counting their agreement as two independent confirmations overstates the evidence, so the system counts independent opinions separately from engines run, and flags any decision carried only by that correlated pair. On the sample document, 4 of 856 confirmed words are in that state.

This is also the argument for architectural diversity over model instance diversity. The question is not how many voters there are, it is how many ways of being wrong they represent.

4 The determinism audit

4.1 Method

The decision function was extracted from the running system and a fixture was generated by calling it, not by reimplementing it: for each of 29,853 token positions in a 41 page United States immigration case index, the candidate readings and the resulting decision were recorded. Deduplicating by candidate set gave 11,780 distinct decisions.

Each was then replayed under every permutation of the order in which candidates are supplied, using `itertools.permutations`, and the set of returned triples (chosen string, rule, backing engines) was required to have cardinality one.

4.2 Result

165 of 11,780 decisions, 1.40 percent, returned more than one distinct outcome depending on ordering alone. Twenty nine changed a digit. Observed pairs include `12-869` against `12369`, `1910` against `1940`, and `11-335` against `11-336`.

Four sites were responsible. All four reduce to the same root cause. Python's `Counter.most_common()` and the built in `max()` are stable: on a tie they return the candidate encountered first, and that order is the insertion order of the mapping the engines were placed in. Nothing about that is a bug in Python. It is a bug in using either function to make a decision that must be reproducible, without saying what happens on a tie.

The fix in each case was to make the tie break a stated total order rather than an accident: count first, then among the tied candidates select by a fixed key, and where no key is defensible, fall through to a refusal instead of a choice. One consequence worth reporting is that 33 of the 165 became refusals, that is, the corrected system declines where the original silently picked.

4.3 What the test could not see

The permutation test covers the voting function. It does not cover the code that calls it. A fifth ordering dependence was found later by an external reviewer in the selection of the page frame, which was resolved alphabetically by engine name. The permutation test could not reach it, because the fixture records decisions and not the frame the decisions were made in.

A sixth was introduced by the author while fixing the fifth, in a metadata field that records which engine supplied the frame, and was found by a further reviewer one commit later. Both are worth reporting because they bound the claim: a permutation test over one function proves a property of that function, and the natural reading of a green test as "the system is deterministic" is wrong.

More generally, permuting insertion order tests insertion order dependence, and not other sources of nondeterminism such as set iteration, floating point summation order, or engine version drift. Property based testing would cover more of that space than exhaustive replay of recorded inputs does.

4.4 Effect on the published accuracy

The obvious objection is that the system's measured accuracy was produced by the code this section shows to be nondeterministic. We re-scored the reference standard after the four fixes. No labelled crop changed its verdict. The 165 affected decisions are concentrated in the contested tail, and the reference standard, being stratified toward contested cases, contains 60 crops of which none fell in that set. This is reassurance rather than proof, and the correct statement is that the fixes did not move the measured number on the sample available.

5 Measurement

5.1 Against the pixels

Sixty token positions were sampled from the index, stratified toward contested decisions, and each was labelled by a human reading the page rendered at 900 dots per inch. Four proved to have an unusable frame and are treated separately in Section 5.2, leaving 56 scored positions.

voter	agrees with the page	counted	95% interval
the PDF's own text layer	46.4%	26 / 56	[34.0, 59.3]
`rapidocr_multi`	78.6%	44 / 56	[66.2, 87.3]
`rapidocr_en`	60.4%	32 / 53	[46.9, 72.4]
the ensemble	**94.6%**	**53 / 56**	**[85.4, 98.2]**

Table 1: Each voter and the ensemble against a human reading of the page. Intervals are Wilson. Note that `rapidocr_en` has a smaller denominator: it returned nothing at all for three of the 56 positions, and a voter that abstains is not scored as wrong. The sample is stratified toward contested decisions, so these are not document level accuracies and must not be read as such.

The comparison that matters in Table 1 is between the last row and the best single row, and it is 16 points. The absolute values are not document accuracy: a random sample of a document where the great majority of tokens are unanimous would spend almost all of its labelling budget confirming easy words, which is why the sample is stratified and why raw percentages from it are not extrapolated without weighting.

Weighted to the document with a Horvitz–Thompson estimator, accuracy is 98.65 percent with three engines and 99.15 percent with four. An exact McNemar test on the discordant pairs gives $p = 0.125$. We report the improvement because it was measured, and we report the p value because at this sample size the improvement cannot be distinguished from chance.

5.2 Four positions the measurement cannot see

Four of the sixty sampled positions have a word box, inherited from the PDF's text layer, that does not correspond to a token. It cuts a glyph, or it frames typographic debris. Every engine then reads inside the same wrong box, and every engine agrees.

This is the important negative result in the paper, and it is structural rather than incidental. A reference standard built by rendering each token's own box and reading it is wrong in exactly the same way the system is wrong, so it cannot detect the error. Agreement based acceptance is blind to any failure the engines share, and a shared frame is the most common way for them to share one. The headline accuracy figures are therefore an upper bound on page level accuracy, not an estimate of it, and the gap between them is unmeasured.

The same reasoning applies to any systematic error common to all engines: a font specific confusion between `l` and `1`, ligature splitting, hyphenation at a line break. Unanimity is evidence only to the extent that the voters fail independently, and we have measured that independence for exactly one pair.

5.3 The reference standard is an instrument

The first version of the labels was read from a contact sheet whose vertical padding caught the neighbouring line. Five of sixty labels were consequently taken from the wrong line. They were found by a rule that is now part of the test suite and runs on every invocation: if every engine agrees and the human disagrees, suspect the human. We recommend it. An unchecked reference standard produces confident errors, exactly as an uncalibrated instrument does.

6 Limitations

We list these in the order we would attack them.

One document, one genre, one language, one scanning lineage from 2002. Nothing here shows that any number transfers.

A single operating point rather than a risk coverage curve. The selective prediction literature reports a curve, and the appropriate comparison for our acceptance rule is a single engine's own confidence thresholded to the same coverage. We have not run it. If the ensemble does not dominate that curve, the premise of the system is weaker than we have argued.

No decision level reference standard. The 56 labelled positions measure whether the chosen string matches the page. They do not measure, per rule, how often a rule that fires is right, except at sample sizes where the intervals span most of the unit interval.

No downstream experiment. The motivating claim is that enumerated uncertainty reduces fabrication when the text reaches a language model. That claim is untested here.

No comparison against an external ensemble. OCR-D's alignment and voting component is available and would be the natural baseline.

The lexicon used by one rule was not audited for overlap with the test document. If it was derived from material that includes it, that rule's accuracy is overstated.

7 Tools used in preparing this work

The software described here was written with substantial assistance from a large language model, and so was this report. The measurements, the fixture, the reference standard and the audit in Section 4 were produced by executing code and recording what it returned, not by asking a model what it thought the answer was. Several of the defects described in Sections 4 and 6, including the fifth ordering dependence and an error in an earlier version of the accuracy accounting, were found by independent language models asked to review the artefact adversarially.

The author is responsible for the entire contents, including every citation, and has checked them.

8 Availability

Code, the five test documents with their SHA-256 checksums and government source URLs, the replay fixture of 11,780 decisions, and the 60 position reference standard with its labels are at `https://github.com/igorsaevets/scanquorum` under the MIT licence. The accuracy figures in Table 1 are recomputed by a test in that repository rather than quoted from a notebook.

References

[1] C. K. Chow. An optimum character recognition system using decision functions. *IRE Transactions on Electronic Computers*, EC-6(4):247–254, 1957.

[2] C. K. Chow. On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1):41–46, 1970.

[3] R. El-Yaniv and Y. Wiener. On the foundations of noise-free selective classification. *Journal of Machine Learning Research*, 11:1605–1641, 2010.

[4] J. G. Fiscus. A post-processing system to yield reduced word error rates: Recognizer Output Voting Error Reduction (ROVER). In *IEEE Workshop on Automatic Speech Recognition and Understanding*, pages 347–354, 1997.

[5] Y. Geifman and R. El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems 30*, pages 4878–4887, 2017.

[6] J. C. Handley. Improving OCR accuracy through combination: a survey. In *IEEE International Conference on Systems, Man, and Cybernetics*, volume 5, pages 4330–4333, 1998.

[7] W. B. Lund and E. K. Ringger. Improving optical character recognition through efficient multiple system alignment. In *Joint Conference on Digital Libraries*, pages 231–240, 2009.

[8] W. B. Lund, D. D. Walker, and E. K. Ringger. Progressive alignment and discriminative error correction for multiple OCR engines. In *International Conference on Document Analysis and Recognition*, pages 764–768, 2011.

[9] T. A. Nartker, S. V. Rice, and S. E. Lumos. Software tools and test data for research and testing of page-reading OCR systems. In *Document Recognition and Retrieval XII*, volume 5676 of *Proceedings of SPIE*, pages 37–47, 2005.

[10] C. Reul, U. Springmann, C. Wick, and F. Puppe. Improving OCR accuracy on early printed books by utilizing cross fold training and voting. In *13th IAPR International Workshop on Document Analysis Systems*, pages 423–428, 2018.

[11] S. V. Rice, J. Kanai, and T. A. Nartker. An algorithm for matching OCR-generated text strings. *International Journal of Pattern Recognition and Artificial Intelligence*, 8(5):1259–1268, 1994.

[12] C. Wick, C. Reul, and F. Puppe. Calamari: a high-performance Tensorflow-based deep learning package for optical character recognition. arXiv:1807.02004, 2018.